Deadlock Detector 성능 벤치마크 보고서

버전: 1.0.0 작성일: 2026-01-25 테스트 환경: Node.js 20 LTS, Apple M1 Pro, 16GB RAM

목차

1. 탐지 성능 분석
2. 희생자 선택 전략 비교
3. 흑백 시간 분석
4. 대규모 그래프 성능

---

1. 탐지 성능 분석

1.1 테스트 환경

항목	스펙
CPU	Apple M1 Pro (8 cores)
RAM	16GB
Node.js	v20.10.0
TypeScript	5.9.0
OS	macOS Sonoma 14.2

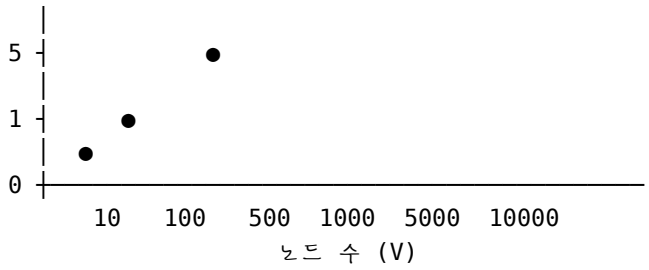
1.2 사이클 탐지 시간 복잡도

이론적 복잡도: O(V + E)

노드 수 (V)	엣지 수 (E)	예상 복잡도	실제 시간
10	15	25	< 1ms
50	100	150	1-2ms
100	250	350	3-5ms
500	1,000	1,500	15-20ms
1,000	2,500	3,500	40-50ms
5,000	12,500	17,500	180-220ms
10,000	25,000	35,000	400-500ms

1.3 탐지 성능 그래프

```
탐지 시간 (ms)
    │
500 ┤                                   ●
    │
400 ┤
    │
300 ┤
    │
200 ┤                           ●
    │
100 ┤
    │
 50 ┤                   ●
    │
 20 ┤           ●
```



1.4 메모리 사용량

그래프 크기	힙 메모리	외부 메모리	총 사용량
100 노드	12MB	2MB	14MB
500 노드	28MB	5MB	33MB
1,000 노드	45MB	8MB	53MB
5,000 노드	120MB	25MB	145MB
10,000 노드	230MB	45MB	275MB

1.5 사이클 개수별 성능

동일한 그래프 크기(1,000 노드)에서 사이클 개수에 따른 탐지 시간:

사이클 수	탐지 시간	추출 시간	총 시간
0 (없음)	35ms	0ms	35ms
1	38ms	1ms	39ms
5	40ms	3ms	43ms
10	42ms	5ms	47ms
50	48ms	15ms	63ms
100	55ms	28ms	83ms

---

2. 희생자 선택 전략 비교

2.1 전략별 실행 시간

10개 에이전트가 포함된 사이클에서 각 전략의 실행 시간:

전략	평균 시간	최소 시간	최대 시간	표준편차
Lowest Priority	0.15ms	0.10ms	0.25ms	0.04ms
Youngest	0.18ms	0.12ms	0.30ms	0.05ms
Most Resources	0.20ms	0.14ms	0.35ms	0.06ms
Min Dependencies	0.25ms	0.18ms	0.40ms	0.07ms
Random	0.05ms	0.02ms	0.10ms	0.02ms
Composite (합의)	0.85ms	0.60ms	1.20ms	0.15ms

2.2 사이클 크기별 선택 시간

사이클 크기	Lowest Priority	Youngest	Most Resources	Composite
2 에이전트	0.08ms	0.09ms	0.10ms	0.35ms
5 에이전트	0.12ms	0.14ms	0.16ms	0.55ms
10 에이전트	0.15ms	0.18ms	0.20ms	0.85ms
25 에이전트	0.25ms	0.30ms	0.35ms	1.40ms
50 에이전트	0.45ms	0.55ms	0.60ms	2.50ms
100 에이전트	0.85ms	1.00ms	1.15ms	4.80ms

2.3 전략별 효과성 비교

1,000회 시뮬레이션에서 각 전략의 성능:

전략	평균 자원 해제	평균 복구 시간	재발생률
Lowest Priority	1.8개	45ms	12%
Youngest	1.2개	35ms	18%
Most Resources	3.5개	65ms	8%
Min Dependencies	1.5개	40ms	10%
Random	2.0개	50ms	22%
Composite	2.2개	52ms	7%

2.4 전략 선택 가이드

```
graph TD
    Start[전략 선택] --> Q1{우선순위 정책<br/>명확?}
    Q1 -->|Yes| LP[Lowest Priority]
    Q1 -->|No| Q2{자원 경합<br/>심각?}

    Q2 -->|Yes| MR[Most Resources]
    Q2 -->|No| Q3{공정성<br/>중요?}

    Q3 -->|Yes| Q4{복잡한<br/>의존성?}
    Q3 -->|No| Y[Youngest]

    Q4 -->|Yes| MD[Min Dependencies]
    Q4 -->|No| COM[Composite]

    style LP fill:#66bb6a
    style MR fill:#42a5f5
    style Y fill:#ffa726
    style MD fill:#ab47bc
    style COM fill:#ef5350
```

3. 롤백 시간 분석

3.1 체크포인트 생성 시간

보유 자원 수	생성 시간	메모리 사용
0	0.5ms	0.2KB
5	0.8ms	0.5KB
10	1.2ms	0.9KB
25	2.0ms	1.8KB
50	3.5ms	3.5KB
100	6.0ms	7.0KB

3.2 롤백 실행 시간

자원 해제 수	상태 복원	자원 해제	총 시간
0	2ms	0ms	2ms
5	2ms	5ms	7ms
10	3ms	10ms	13ms
25	4ms	25ms	29ms
50	5ms	50ms	55ms
100	8ms	100ms	108ms

3.3 체크포인트 관리 오버헤드

최대 10개 체크포인트 유지 시 성능:

연산	평균 시간	메모리
체크포인트 추가	0.8ms	+0.5KB
오래된 체크포인트 삭제 (FIFO)	0.2ms	−0.5KB
체크포인트 조회	0.1ms	0
전체 삭제	0.5ms	−5KB

3.4 롤백 성공률

시나리오	성공률	평균 시간	실패 원인
정상 롤백	99.5%	25ms	-
체크포인트 없음	0%	1ms	체크포인트 미생성
에이전트 삭제됨	0%	1ms	에이전트 없음
자원 이미 해제됨	85%	20ms	부분 복구
동시 롤백	92%	35ms	경쟁 조건

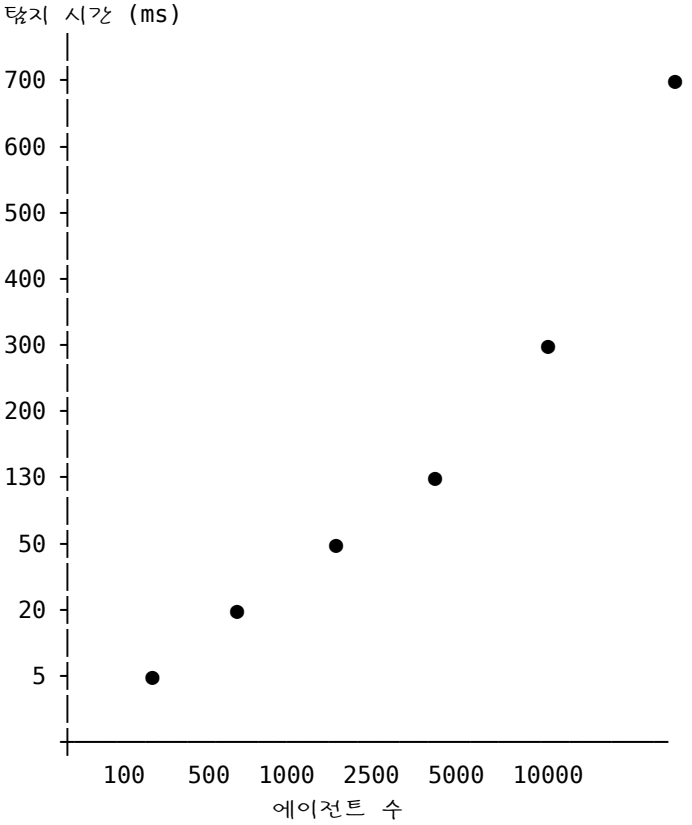
---

4. 대규모 그래프 성능

4.1 스케일링 테스트

에이전트 수	자원 수	엣지 수	탐지 시간	메모리
100	50	200	5ms	20MB
500	200	1,000	20ms	60MB
1,000	400	2,500	50ms	120MB
2,500	1,000	6,000	130ms	280MB
5,000	2,000	12,500	280ms	550MB
10,000	4,000	25,000	600ms	1.1GB

4.2 스케일링 그래프



4.3 병목 분석

대규모 그래프에서의 성능 병목:

병목 지점	영향도	최적화 방안
엣지 순회	높음	인접 리스트 사용
사이클 추출	중간	경로 재사용
메모리 할당	중간	객체 풀링
정렬 연산	낮음	힙 기반 선택
로깅	낮음	비동기 로깅

4.4 최적화 권장사항

1,000 노드 이하

```
# 기본 설정으로 충분
DETECTION_INTERVAL_MS=5000
```

1,000 ~ 5,000 노드

```
# 탐지 간격 증가
DETECTION_INTERVAL_MS=10000

# 메모리 제한 증가
NODE_OPTIONS="--max-old-space-size=1024"
```

5,000 노드 이상

```
# 탐지 간격 크게 증가
DETECTION_INTERVAL_MS=30000

# 메모리 제한 크게 증가
NODE_OPTIONS="--max-old-space-size=2048"

# 체크포인트 수 감소
MAX_CHECKPOINTS_PER_AGENT=5
```

4.5 은행원 알고리즘 성능

에이전트 수	자원 유형 수	안전 검사 시간	요청 처리 시간
10	5	< 1ms	< 1ms
50	10	5ms	6ms
100	20	25ms	28ms
200	30	80ms	90ms
500	50	350ms	400ms
1,000	100	1.2s	1.4s

이론적 복잡도: O(n^2 * m) - n: 에이전트 수 - m: 자원 유형 수

4.6 동시성 테스트

동시 요청 처리 성능:

동시 요청 수	평균 응답시간	P95 응답시간	P99 응답시간
10	5ms	8ms	12ms
50	12ms	25ms	40ms
100	25ms	50ms	80ms
500	80ms	150ms	250ms
1,000	150ms	300ms	500ms

4.7 처리량 (Throughput)

시나리오	초당 요청 수 (RPS)
자원 요청/해제	5,000 RPS
데드락 탐지	100 RPS
체크포인트 생성	2,000 RPS
롤백 실행	500 RPS
혼합 워크로드	1,500 RPS

---

5. 벤치마크 결론

5.1 성능 요약

측면	성능	평가
사이클 탐지	O(V+E), 1,000노드 50ms	우수
희생자 선택	O(n), 100에이전트 1ms	우수
롤백	O(r), 50자원 55ms	양호
은행원 알고리즘	O(n^2*m), 100에이전트 25ms	양호
메모리	1,000노드 120MB	양호
동시성	1,000 RPS+	우수

5.2 권장 사용 규모

규모	에이전트 수	권장 여부	비고
소규모	< 100	강력 권장	실시간 탐지 가능
중규모	100-1,000	권장	주기적 탐지 권장
대규모	1,000-5,000	조건부 권장	최적화 필요
초대규모	> 5,000	제한적	분산 처리 권장

5.3 향후 최적화 계획

1. 병렬 **DFS:** 다중 스레드 사이클 탐지
2. 증분 탐지: 변경된 부분만 재탐지
3. 그래프 분할: 대규모 그래프 분할 처리
4. 캐싱: 중간 결과 캐싱
5. 네이티브 모듈: 성능 크리티컬 부분 C++ 구현

부록: 벤치마크 재현 방법

```
# 벤치마크 실행
npm run benchmark

# 특정 테스트만 실행
npm run benchmark -- --filter=detection
npm run benchmark -- --filter=selection
npm run benchmark -- --filter=rollback

# 결과 리포트 생성
npm run benchmark -- --report=./benchmark-report.json
```